<?php

MATERIAL DE ESTUDIO / DESARROLLO WEB

UNIDAD DIDÁCTICA 02

Tutorial 04
Carrito de compra

Cookies, catálogo y conservación del carrito

PHP · Cookies · Formularios · Carrito

APUNTES DE CLASE

Desarrollo web · PHP · Tutorial guiado

{ PHP }

UD 02 · DWES · PHP 8.1+

Tutorial 04 · Carrito de compra con cookies

Objetivo: Vamos a crear un catálogo pequeño, indicar cantidades, guardar la selección durante una semana y recuperarla cuando se vuelva a abrir la página. El navegador almacena cantidades en una cookie; los precios siempre proceden del catálogo del servidor.

Antes de comenzar

Utiliza PHP 8.1 o posterior y un servidor web local (por ejemplo, Apache de XAMPP). Crea una carpeta de proyecto dentro de `htdocs/UD2/` y **escribe los archivos con el contenido mostrado en cada paso**. El directorio `Codigo/` contiene esos mismos archivos listos para ejecutar.

Crea `htdocs/UD2/Carrito/` con cinco archivos: `Producto.php`, `productos.php`, `index.php`, `guardar.php` y `borrar.php`. La página de entrada será `http://localhost/UD2/Carrito/index.php`.

Paso 1. Definir la clase `Producto`

Crea `Producto.php`. Cada producto tiene un nombre y un precio privados. El constructor recibe los dos valores y los métodos `getNombre()` y `getPrecio()` los devuelven. Los métodos no generan HTML ni modifican las cookies.

Archivo `Codigo/Producto.php`

```php
<?php
declare(strict_types=1);

class Producto
{
    public function __construct(
        private string $nombre,
        private float $precio
    ) {}

    public function getNombre(): string
    {
        return $this->nombre;
    }

    public function getPrecio(): float
    {
        return $this->precio;
    }
}
```

La clase `Producto` es el modelo de datos. Más adelante, cuando calculemos el total, obtendremos el precio mediante `getPrecio()`.

Paso 2. Crear el catálogo del servidor

Crea `productos.php`. Este archivo importa `Producto.php` y prepara un array asociativo `$productos`. Cada clave (`P01`, `P02`…) identifica de forma estable a un producto y cada valor es una instancia de la clase.

El precio se escribe una sola vez en el catálogo; no se envía en ningún campo oculto del formulario para recuperarlo después. Solo las cantidades llegarán desde el navegador.

Archivo `Codigo/productos.php`

```php
<?php
declare(strict_types=1);
require_once __DIR__ . '/Producto.php';

$productos = [
    'P01' => new Producto('Pan integral', 1.50),
    'P02' => new Producto('Café molido', 4.80),
    'P03' => new Producto('Leche', 1.15),
    'P04' => new Producto('Queso', 3.20)
];
```

Por ejemplo, `$productos['P02']->getPrecio()` vale `4.80`. Esa cantidad la controla el servidor, no una cookie ni una entrada del formulario.

Paso 3. Mostrar el formulario y calcular el total

Crea `index.php`. Primero carga el catálogo y define la función `h()` para escapar texto HTML. La lectura de `$_COOKIE['carrito_demo']` recupera una cadena JSON; `json_decode(..., true)` la transforma en un array asociativo. Si la cookie no existe o el JSON no es un array válido, se utiliza un carrito vacío.

Luego se recorren los productos del catálogo. Para cada código se obtiene la cantidad correspondiente de la cookie, se valida que sea un entero entre 0 y 20 y se suma `cantidad × precio` al total. Si una cantidad de la cookie no es válida, se trata como cero.

El HTML genera un `input` por producto. Un nombre como `cantidad[P01]` crea en PHP el array `$_POST['cantidad']['P01']`. El valor inicial del input procede de la cookie validada.

Archivo `Codigo/index.php`

```php
<?php
declare(strict_types=1);
require_once __DIR__ . '/productos.php';

function h(string $valor): string
{
    return htmlspecialchars($valor, ENT_QUOTES, 'UTF-8');
}

$carrito = json_decode($_COOKIE['carrito_demo'] ?? '{}', true);
if (!is_array($carrito)) {
    $carrito = [];
}
$total = 0.0;
foreach ($productos as $id => $producto) {
    $cantidad = filter_var(
        $carrito[$id] ?? 0,
        FILTER_VALIDATE_INT,
        ['options' => ['min_range' => 0, 'max_range' => 20]]
    );
    $total += ($cantidad === false ? 0 : $cantidad) * $producto->getPrecio();
}
?>
<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Carrito con cookies</title>
</head>
<body>
```

```
    <h1>Carrito con cookies</h1>
    <form action="guardar.php" method="post">
        <?php foreach ($productos as $id => $producto): ?>
            <?php
            $cantidad = filter_var(
                $carrito[$id] ?? 0,
                FILTER_VALIDATE_INT,
                ['options' => ['min_range' => 0, 'max_range' => 20]]
            );
            ?>
            <label>
                <?= h($producto->getNombre()) ?>
                (<?= number_format($producto->getPrecio(), 2, ',', '.') ?> €)
                <input type="number" name="cantidad[<?= h($id) ?>]"
                       min="0" max="20"
                       value="<?= $cantidad === false ? 0 : $cantidad ?>">
            </label>
            <br>
        <?php endforeach; ?>
        <button type="submit">Guardar cantidades</button>
    </form>
    <p>Total: <?= number_format($total, 2, ',', '.') ?> €</p>
    <form action="borrar.php" method="post">
        <button type="submit">Vaciar carrito</button>
    </form>
</body>
</html>
```

El navegador ve los nombres y precios para mostrarlos, pero `guardar.php` no aceptará precios enviados por el cliente. El total tampoco sale directamente del JSON: se recalcula usando las instancias de `Producto`.

Paso 4. Guardar las cantidades en una cookie

Crea `guardar.php`. Se ejecuta cuando se pulsa «Guardar cantidades». Exige el método `POST`, verifica que `cantidad` sea un array y recorre únicamente los códigos existentes en `$productos`. Cada cantidad se valida con `FILTER_VALIDATE_INT`, admitiendo de 0 a 20. Los ceros no se incluyen en el JSON para que ocupe menos.

`json_encode()` transforma el carrito en una cadena JSON. `setcookie()` envía al navegador una cookie que caduca en siete días; `httponly` impide leerla desde JavaScript y `samesite=Lax` limita ciertos envíos entre sitios. Finalmente se redirige a `index.php` para mostrar el carrito actualizado.

Archivo `Codigo/guardar.php`

```
<?php
declare(strict_types=1);
require_once __DIR__ . '/productos.php';

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    exit;
}

$datos = $_POST['cantidad'] ?? [];
if (!is_array($datos)) {
    http_response_code(400);
    exit('Las cantidades deben enviarse en un array.');
}

$carrito = [];
foreach ($productos as $id => $producto) {
    $cantidad = filter_var(
        $datos[$id] ?? 0,
        FILTER_VALIDATE_INT,
```

```
        ['options' => ['min_range' => 0, 'max_range' => 20]]
    );
    if ($cantidad === false) {
        http_response_code(400);
        exit('Cada cantidad debe ser un entero de 0 a 20.');
    }
    if ($cantidad > 0) {
        $carrito[$id] = $cantidad;
    }
}

setcookie('carrito_demo', json_encode($carrito, JSON_THROW_ON_ERROR), [
    'expires' => time() + 7 * 86400,
    'path' => '/',
    'secure' => !empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off',
    'httponly' => true,
    'samesite' => 'Lax'
]);
header('Location: index.php');
exit;
```

La función `setcookie()` envía una cabecera HTTP y por eso se llama antes de mostrar HTML. La cookie no es un lugar seguro para guardar información que requiera confianza: el usuario puede modificarla, de modo que siempre validamos su contenido.

Paso 5. Vaciar el carrito

Crea `borrar.php`. El botón de vaciado envía un `POST` a este archivo. Para eliminar la cookie, se utiliza el mismo nombre y ruta pero con una fecha de caducidad ya pasada. Después vuelve a `index.php`.

Archivo `Codigo/borrar.php`

```
<?php
declare(strict_types=1);

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    http_response_code(405);
    exit;
}

setcookie('carrito_demo', '', [
    'expires' => time() - 3600,
    'path' => '/',
    'secure' => !empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off',
    'httponly' => true,
    'samesite' => 'Lax'
]);
header('Location: index.php');
exit;
```

Si no utilizases la misma ruta (`path`) al eliminar la cookie, el navegador podría mantener la cookie original. Por eso `guardar.php` y `borrar.php` usan ambos `path => '/'`.

Resultado y comprobación

Abre `index.php`: verás cuatro productos con cantidad inicial 0 y total 0,00 €. Coloca 2 unidades de pan integral y 1 de café molido. Tras guardar, el total será **7,80 €** (2 × 1,50 + 4,80) y las cantidades seguirán apareciendo al recargar.

Pulsa «Vaciar carrito»: vuelven los ceros y el total 0,00 €. Una cantidad negativa o mayor que 20 será rechazada por el servidor, incluso si se manipula el HTML. Un código que no figure en el catálogo será ignorado. Este ejemplo sirve para aprender cookies, no para validar pagos ni pedidos reales.